

老賈操控 Odoo 能力總盤點 + 不可控項替代方案

參照 [API 需求對照矩陣](#)再往前推——Claude Code(老賈)實際能操控什麼、做到什麼程度、碰壁的每一項有什麼替代解 | 官方原始碼查證 (含 safe_eval 逐行反證) | 2026-07-22 (Odoo 18.0 自架 Community) | 如菓工程統籌

一句話回答：Odoo 幾乎所有「營運與內容」功能老賈都能操控，因為整套系統跑在同一組 API 上、任何公開方法都能遠端呼叫。碰壁的不是「功能」而是三類邊界——①核心層擋住的私有方法（走公開 wizard 繞）②Odoo 主動通知老賈（要小繞道）③少數必須人工一次性取得的憑證（金流帳密/DNS/SSL）。每一個都有已查證的可行替代解，沒有死路。

操控分三級：全自動 老賈 API 直接做；半自動 要走 wizard/自訂小工具/輪詢；需人工一次 取得憑證後老賈接管。VETO 九項需求 + 日常營運，落在「全自動」的佔絕大多數。

全自動為主

營運 + 內容層操控

3 類邊界

私有方法 / 事件推播 / 人工憑證

0 死路

每個缺口都有查證過的替代解

18.0

原始碼逐行查證 (含 safe_eval)

1. 老賈可操控功能完整清單 (含分級)

分級：全自動 老賈用 API 直接執行 | 半自動 要走 wizard 或自訂小工具 | 需人工一次 取得憑證/簽約後老賈接管。以下為 VETO 會用到的模組，非窮舉——Odoo 40+ 模組全是同一套 API，原則上都可操控。

功能領域	分級	老賈實際能做 (模型 / 方法)
商品管理	全自動	上下架、改價、改文案圖片、變體、分類、SEO、庫存綁定 (product.template/product.product CRUD + is_published)
B2B 經銷帳號	全自動	建經銷公司 + 聯絡人、開 portal 登入帳號、指派分級價 (res.partner / portal.wizard.user.action_grant_access)
分級批發價	全自動	建價格表、量價階梯、檔期價、指派客戶 (product.pricelist.item : min_quantity/fixed_price)

訂單處理	全自動	建單、確認、改單、取消、退款 (<code>sale.order.action_confirm</code>)
賒銷 / 月結條件	全自動	設 Net 30、信用額度、讀即時應收 (<code>account.payment.term / res.partner.credit_limit</code>)
出貨 / 庫存	全自動	建出入庫單、驗收出貨、庫存查詢、跨點調撥 (<code>stock.picking.button_validate / stock.quant</code>)
進貨採購	全自動	建採購單、確認、收貨 (<code>purchase.order.button_confirm</code>)
條碼盤點	全自動	存原廠 UPC、程式化盤點調整 (<code>stock.quant.inventory_quantity + action_apply_inventory</code>)
會計過帳	全自動	建分錄、發票、過帳、讀總帳 (<code>account.move.action_post / account.move.line</code>)
應收沖帳	半自動	登記收款 + 自動沖帳 (<code>account.payment.register.action_create_payments</code> ，帶來源發票 context) ——走 wizard 非直呼
開電子發票	半自動	觸發開立送綠界、作廢、折讓 (<code>account.move.send.wizard.action_send_and_print</code>) ——送出底層方法私有、走公開 wizard
網站內容 (文案/圖/選單)	全自動	改頁面文案/圖/選單、新增頁面、部落格、SEO meta (<code>ir.ui.view.arch / website.page / website.menu / ir.attachment</code> 全 API 可寫) ——深度見 §2
前台版面排版 (改現有積木)	全自動	改 section 的欄位/間距/顏色/圓角 = 改 arch 裡的 Bootstrap class (API write <code>ir.ui.view.arch</code>) ——§2 詳解 + 三個雷
主題色/字體/圓角	半自動	設計 tokens 是 SCSS 變數，存成 <code>ir.attachment + ir.asset</code> ，API 可改 (見 §2)
CRM / 行銷	全自動	商機、客戶分群、Email 行銷、自動化行銷 (<code>crm.lead / mailing.mailing</code> 等模型 CRUD)
報表數據匯出	全自動	任意模型 <code>search_read / read_group</code> 撈數據自算 (銷售/庫存/帳齡分析)
退貨 / 退款	全自動	退貨單、貸項通知單 (<code>stock return / account.move reversal</code>)
LABAVO 品牌主題 (整套版型)	半自動	整套客製 = 寫 theme module / snippet 程式碼 + Git 部署 (自架/Odoo.sh，檔案層) ——§2 建議走這條而非狂改 arch

經銷專屬品項	半自動	原生無「按客戶隱藏商品」——老賈用 API 建一條 record rule 解 (見 §4)
月結對帳單	半自動	現成報表 Enterprise-only——老賈用 API 撈資料自產 PDF (見 §4)

2. 前台 UI/UX 與版面排版操控深度 (老闆追問補查·反向查核過)

老闆問「前台網站 UI/UX 與版面設計排版，老賈能不能操控修改運作」——答案是**能**，分三個層次，全部經官方原始碼查證；但「直接改版面資料」有三個實作雷，我反向查核後老實列出，不對你打包票「隨便改都沒事」。

三個層次的操控 (都可行)

層次	分級	老賈實際怎麼做 (機制查證)
① 改現有版面/內容	全自動	每個拖拉積木在資料層就是 <code>ir.ui.view.arch</code> 裡一段 <code><section></code> + Bootstrap class——改欄寬(<code>col-lg-*</code>)、間距(<code>pt/pb</code>)、顏色(<code>class</code>)、圓角(<code>rounded</code>) = 改這段 HTML 的 <code>class</code> 字串，老賈 API <code>write arch</code> 即可。文字/圖/連結：靜態的直接寫 <code>arch</code> ；綁資料的 (<code>t-field data-oe-model/field</code>) 直接 <code>write</code> 目標欄位 (如 <code>blog.post.content</code>)
② 主題色/字體/圓角	半自動	設計 tokens 是 SCSS 變數 (5 主題色 <code>o-color-1~5</code> 、字體、 <code>btn-border-radius</code>) ——客製後由 <code>web_editor.assets</code> 存成一筆自訂 SCSS 的 <code>ir.attachment</code> + 一筆 <code>ir.asset(replace)</code> ，老賈可 API CRUD 這兩個模型改主題 (反證修正：不是存在 <code>ir.ui.view</code> ，是 13+ 版的資產系統)
③ 做 LABAVO 自訂積木	半自動	把 LABAVO 特殊版型做成可重用積木 = 寫 <code>s_</code> 前綴 QWeb 樣板 + 註冊進 <code>website.snippets</code> 面板 (<code>t-snippet/t-thumbnail</code>)，SCSS/JS 隨 module 打包；加前台 JS/CSS 走 <code>ir.asset + web.asset s_frontend bundle</code> ——官方教學支援的正規路，自架/Odoo.sh 才能裝

⚠ 反向查核：直接改 `arch` 的三個雷 (防過度承諾)

層次①「API write arch 改版面」技術上成立，但反證員從原始碼挖出三個必須知道的陷阱——所以「日常小改（換文案/圖/顏色）用 API 沒問題，但整套品牌版型正規做法是走 §③ 積木/主題程式碼，不是狂改 arch」：

1. **升級漂移**：任何對 arch 的改動（含 API write）會把該 view 的 `arch_updated` 設 True——之後模組升級**不再用官方 XML 覆蓋這個 view**：你的客製保留了，但也**再也吃不到官方後續的版面修正**，要回官方版得手動 `reset_arch`。改越多、升級時要顧的越多。
2. **COW 陷阱**：改一個「共用版型」時，帶 `website_id context` = 複製出網站專屬副本再改（不影響別站）；不帶 = **直接改到通用 view、影響全部網站**。老賈 API 直寫要清楚在改哪一份，否則可能動到不該動的。
3. **編輯器辨識**：拖拉編輯器靠 `data-snippet` / `s_ class` / `data-name` 認出「這是積木」。老賈 API 手寫塞進 arch 的 HTML 若缺這些屬性，**回編輯器會辨識不出是積木**（不會弄壞頁面，但失去可視化拖拉編輯）——要保留可編輯性，寫入的 HTML 得沿用官方積木的屬性結構。

務實結論：前台 UI/UX 老賈全能操控——日常內容/小版面調整走 API（全自動），品牌級版型/主題走「寫 snippet+主題程式碼 Git 部署」（半自動、避開三個雷、可版控可回滾）。設計定案由你拍板，實作與後續調整交老賈，切得乾淨。

來源：github.com/odoo/odoo 18.0 原始碼（`website/views/snippets/s_text_image.xml` / `web_editor/models/ir_ui_view.py` / `web_editor/models/assets.py` / `website/static/src/scss/primary_variables.scss` / `base/models/ir_qweb_fields.py` / `website/models/ir_ui_view.py` 的 COW 與 `arch_updated` 邏輯）、官方 Developer Tutorials「Build a website theme」snippet 章、`ir.asset/assets bundle` 文件——關鍵主張反證：改 arch 改版面 CONFIRMED、主題 token 存 `ir.attachment+ir.asset` CONFIRMED、「直改 arch 都沒事」REFUTED（三個雷）。

3. 操控的真正邊界（三類，及為什麼）

邊界一：私有方法——核心層的鐵牆

Odoo 核心（`odoo/service/model.py` 的 `get_public_method`，前份已反證）擋下所有底線開頭 / `@api.private` 方法的遠端呼叫。**這是好事**——AI 只能碰官方設計成可對外的公開商業方法（`action_confirm/action_post/button_validate` 全公開）。遇到私有方法（開發票

_create_invoices、送綠界_l10n_tw_edi_send) 就走它的公開 wizard 包裝——每個都有正確入口，寫在 §1 表上。半自動

邊界二：Odoo 主動通知老賈——要小繞道 (本次最深查證)

「發生 X 就自動叫老賈」這件事，Odoo 18.0 沒有原生出站 webhook，而且反證員逐行確認：server action 的 Python 沙箱 (safe_eval) 硬擋 import、context 裡也沒有任何 HTTP 工具 (黑名單封 IMPORT_NAME 位元碼 + dunder 名稱檢查封 __import__，兩條路都死) ——所以「server action 直接發 HTTP 打老賈」官方手段做不到。三條替代路，已排序：

路徑	可行	做法	取捨
(c) 老賈端輪詢 (預設推薦)	✓ 最實際	老賈排程 (工作排程器 + claude -p) 定時跑 xmlrpc search_read 抓新單 (domain 依 create_date)	零 Odoo 側程式碼；有 1-5 分鐘輪詢延遲
(b) 自訂 module 方法	✓ 要即時才用	自架加一個小 addon 的公開 model 方法 (正常 Python 可 import requests 發 HTTP) → base.automation 綁 create/write 觸發	真正即時零延遲；維護成本 = 一顆 module (自架允許)
(a) server action 直發 HTTP	✗ 官方手段不可行	safe_eval 封死 import 與 HTTP 工具	唯一未支援側門 (__import__ 已載入模組) 脆弱且屬安全弱點，NEVER 用

結論：VETO 日常事件用 (c) 輪詢就夠 (訂單進來 1-5 分鐘內老賈就知道) ；要「秒級即時」才升級 (b) 自訂 module。

邊界三：必須人工一次性取得的憑證——之後老賈全接管

這類不是「AI 能力不足」，是現實世界要人簽名/驗證的事：向綠界簽約取得 MerchantID/HashKey、金流商帳密、網域 DNS 設定、SSL 憑證核發、API 金鑰索取。查證確認：這些憑證一旦拿到，填入 Odoo (res.config.settings write)、啟用金流商

(`payment.provider.state` 改 `enabled` , API 可寫) 、產對帳單——全部老賈接管。分界線清楚：取得 = 人工一次，之後 = 全自動。 (需人工一次)

4. 兩個硬傷的替代解 (已查證落地細節)

硬傷① 經銷專屬品項可見性

最實際解 = 老賈用 API 建一條 `record rule` (`ir.rule`) : 綁「經銷商」`res.groups` 群組，用 `domain` 過濾 `product.template`——這是 Odoo 原生的權限機制，後台與 `portal` 都生效，能做到真正「按客戶群隔離商品」。需要「寫」的只是一條 `domain`，不是開發整包模組，且 `ir.rule/res.groups` 都是常規模型、老賈能直接 API 建立。前台可再疊多網站 (B2B 站 + 選單 `visibility` 設「需登入 `connected`」) 做雙保險。 (半自動·一次性設定)

為什麼原生不行：eCommerce 商品層沒有「按角色/群組可見性」選項 (官方文件確認) ；`website_id` 只能一站或全站、無法按客戶群。所以走 `record rule`。

硬傷② 月結對帳單 / 帳齡報表

現成報表 (`Partner Ledger` / `Aged Receivable` / `Follow-up`) 是 Enterprise-only，但 **Community** 自建完全可行、不需 Enterprise、不需在 Odoo 內寫程式：反證確認 `account.move.line` 有對帳單需要的全部欄位 (`partner_id` / `date_maturity` 到期日 / `debit` / `credit` / `amount_residual` 未沖銷餘額 / `reconciled` / `move_id`) ——老賈用 `search_read` / `read_group` 撈出來、依 `date_maturity` 分帳齡桶，在老賈這端自產對帳單 PDF (版面完全照 VETO 需求做，比 Enterprise 制式報表更貼)。 (半自動·老賈自建)

5. VETO 落地行動

1. 部署 = 自架 **Community** (呼應前份報告) : 授權費 0、含綠界電子發票模組、API 免費、`record rule/theme` 全自由。老賈用 API key 經 `execute_kw` 或自建 MCP 操控。

2. 事件通知先用輪詢 (c)：零 Odoo 側程式碼，訂單/收款進來老賈定時抓。日後要秒級即時再加一顆自訂 module (b)。
3. 硬傷用 record rule + 自建對帳單解，兩者都是老賈一次性設定/開發，不必買 Enterprise。
4. 人工項清單交老闆：綠界簽約取金鑰、網域 DNS、SSL——這幾件你/我協作辦一次，之後系統營運全交老賈。
5. 一個待確認：綠界電子發票模組 `l10n_tw_edi_ecpay` 的授權版別（前份查證指公開 Community 庫，本次補查未能從私有 Enterprise repo 反面確證）——開發店實測時第一件驗。

6. 查證方法與誠信聲明

- 本報告 = 既有 API 需求對照矩陣（7 直接/2 硬傷/私有方法邊界）+ 本次兩路替代方案補查（事件驅動、硬傷落地、人工項）綜合。
- 最硬核的一條——「server action 不能直發 HTTP」——反證員逐行核對 `safe_eval.py` 原始碼（`_BLACKLIST` 封 `IMPORT_NAME`、dunder 名稱檢查封 `_import_`、eval context 無 HTTP 工具），CONFIRMED；替代路徑 (b)(c) 亦以官方原始碼佐證。
- 第二路（硬傷替代）首輪回傳過短被品管防呆攔下，已重派 opus 補查——record rule 與 `account.move.line` 欄位全從官方 18.0 原始碼確認。
- 誠實邊界：分級（全自動/半自動/需人工）依「老賈能否純 API 完成」判定；綠界模組授權版別列為唯一待實測項，不假裝已確認。
- 2026-07-23 補：§2 前台 UI/UX 操控深度（老闆看報告後指出原版只帶一行）——兩路補查、原始碼逐檔查證；反向查核抓到「直改 `arch` 三個雷」（升級漂移 `arch_updated` / COW / 編輯器辨識），並修正一個假設（主題色存 `ir.attachment+ir.asset` 非 `ir.ui.view`）——如實補上，不對前台操控打包票。

資料來源：github.com/odoo/odoo 18.0 原始碼（`service/model.py` / `tools/safe_eval.py` / `ir_actions.py` / `ir_cron.py` / `account_move_line.py` / `payment_provider.py` / `website_page.py` 等逐檔）、odoo.com/documentation（`external_api` / `security record rules` / `ecommerce` / Taiwan localization）、apps.odoo.com——查證日 2026-07-22。方法/欄位名以 Odoo 18.0 為準。「需人工」「待實測」如實標註，NEVER 編造。

本報告為內部技術決策文件。 | 賈維斯 | 2026-07-22 | VETO 評估系列第十二份：接續 `veto-odoo-api-matrix`（需求對照）、`veto-odoo-fullsite-ai`（三層操控）